Application Programming Interface

ECE 2564, Embedded Systems

Fall 2025

Application Programming Interface (API)

- An API is a mechanism that allows two software components to communicate with each other
- A set of definitions and protocols enables this communication
- An API can hide the internal details of how a system works exposing those useful parts to the programmer
- An API opens a software system to interactions from the outside
- APIs are usually provided by the hardware manufacturer

- For our MSP432 there are two different types of API
  - HAL = Hardware Abstraction Layer
  - DriverLib = Driver Library (gpio.c and gpio.h)

How Difficult was our Blink Program to Write?

- Was it easy to write?
- Could errors easily get into the code?
- Can it be debugged? Tested? Maintained?
- Is the code portable?

Our Original Blink Program

```
#define LED1_MASK 1
#define S1_MASK 2
int main(void){
    // set up the register addresses for Port 1
    unsigned char *P1SEL0_addr =  (unsigned char *) 0x40004c0a;
    unsigned char *P1SEL1_addr =  (unsigned char *) 0x40004c0c;
    unsigned char *P1DIR_addr =  (unsigned char *) 0x40004c04;
    unsigned char *P1REN_addr =  (unsigned char *) 0x40004c06;
    unsigned char *P1IN_addr =  (unsigned char *) 0x40004c00;
    unsigned char *P1OUT_addr =  (unsigned char *) 0x40004c02;
    // configure LED1 bit for digital output (P1.0)
    *P1SEL0_addr &= ~LED1_MASK;
    *P1SEL1_addr &= ~LED1_MASK;
    *P1DIR_addr |= LED1_MASK;
    // configure switch S1 for digital input (P1.1)
    *P1SEL0_addr &= ~S1_MASK;
    *P1SEL1_addr &= ~S1_MASK;
    *P1DIR_addr &= ~S1_MASK;
    *P1REN_addr |= S1_MASK;
    *P1OUT_addr |= S1_MASK;
    // loop forever
    while(1){
        if ((*P1IN_addr & S1_MASK) == 0)
            *P1OUT_addr |= LED1_MASK;
        else
            *P1OUT_addr &= ~LED1_MASK;
    }
}
```

Same Functionality with DriverLib

```
// MSP432 example: pressing switch 1 causes LED1 to turn ON
#include<ti/devices/msp432p4xx/driverlib/driverlib.h>
#define PRESSED 0

int main(void)
{
   // step 1: Stop the watchdog timer (will explain later)
   WDT_A_hold(WDT_A_BASE);

   // step 2: Initializing LED1, which is on Pin 0 of Port P1
   GPIO_setAsOutputPin(GPIO_PORT_P1, GPIO_PIN0);

   // step 3: Initializing S1, which is on Pin 1 of Port 1
   GPIO_setAsInputPinWithPullUpResistor (GPIO_PORT_P1, GPIO_PIN1);

   while (1) {

     // If the button is not pressed, keep the LED off
     if (GPIO_getInputPinValue(GPIO_PORT_P1, GPIO_PIN1) != PRESSED)
        GPIO_setOutputLowOnPin(GPIO_PORT_P1, GPIO_PIN0);
     // otherwise, turn the LED on
     else
        GPIO_setOutputHighOnPin(GPIO_PORT_P1, GPIO_PIN0);
    }
}
```

Comparison of Code

- Both of these lines of code turn ON an LED
- Which one is easier to write, debug, read, etc.?

```
*P1OUT_addr |= LED1_MASK;
```

```
GPIO_setOutputHighOnPin(GPIO_PORT_P1, GPIO_PIN0);
```

MSP432 DriverLib



- **USER's GUIDE**
- "The Texas Instruments MSP432 Driver Library (DriverLib) is a set of fully functional APIs used to configure, control, and manipulate the hardware peripherals of the MSP432 platform."

GPIO Functions in DriverLib[1]

- void GPIO_clearInterruptFlag (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_disableInterrupt (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_enableInterrupt (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- uint_fast16_t GPIO_getEnabledInterruptStatus (uint_fast8_t selectedPort)
- uint8_t GPIO_getInputPinValue (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- uint_fast16_t GPIO_getInterruptStatus (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_interruptEdgeSelect (uint_fast8_t selectedPort, uint_fast16_t selectedPins, uint_fast8_t edgeSelect)
- void GPIO_registerInterrupt (uint_fast8_t selectedPort, void(*intHandler)(void))
- void GPIO_setAsInputPin (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_setAsInputPinWithPullDownResistor (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_setAsInputPinWithPullUpResistor (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_setAsOutputPin (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_setAsPeripheralModuleFunctionInputPin (uint_fast8_t selectedPort, uint_fast16_t selectedPins, uint_fast8_t mode)
- void GPIO_setAsPeripheralModuleFunctionOutputPin (uint_fast8_t selectedPort, uint_fast16_t selectedPins, uint_fast8_t mode)
- void GPIO_setDriveStrengthHigh (uint_fast8_t selectedPort, uint_fast8_t selectedPins)
- void GPIO_setDriveStrengthLow (uint_fast8_t selectedPort, uint_fast8_t selectedPins)
- void GPIO_setOutputHighOnPin (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_setOutputLowOnPin (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_toggleOutputOnPin (uint_fast8_t selectedPort, uint_fast16_t selectedPins)
- void GPIO_unregisterInterrupt (uint_fast8_t selectedPort)

GPIO_setOutputHighOnPin Example[2]

void GPIO_setOutputHighOnPin (uint_fast8_t *selectedPort*, uint_fast16_t *selectedPins*)

This function sets output HIGH on the selected Pin.

This function sets output HIGH on the selected port's pin.

Parameters

selectedPort	is the selected port. Valid values are: ■ **GPIO_PORT_P1** ■ **GPIO_PORT_P2** ■ **GPIO_PORT_P3** ■ **GPIO_PORT_P4** ■ **GPIO_PORT_P5** ■ **GPIO_PORT_P6** ■ **GPIO_PORT_P7** ■ **GPIO_PORT_P8** ■ **GPIO_PORT_P9** ■ **GPIO_PORT_P10** ■ **GPIO_PORT_P11** ■ **GPIO_PORT_PJ**

[2] Section 11.4.2.17, Page 141 DriverLib User Guide

GPIO_setOutputHighOnPin Example[3]

selectedPins	is the specified pin in the selected port. Mask value is the logical OR of any of the following: ■ **GPIO_PIN0** ■ **GPIO_PIN1** ■ **GPIO_PIN2** ■ **GPIO_PIN3** ■ **GPIO_PIN4** ■ **GPIO_PIN5** ■ **GPIO_PIN6** ■ **GPIO_PIN7** ■ **GPIO_PIN8** ■ **GPIO_PIN9** ■ **GPIO_PIN10** ■ **GPIO_PIN11** ■ **GPIO_PIN12** ■ **GPIO_PIN13** ■ **GPIO_PIN14** ■ **GPIO_PIN15**

Modified bits of **PxOUT** register.

Returns

None

MSP432 DriverLib Advantages

- The MSP432 DriverLib is a software layer that uses intuitive function calls replacing direct register accesses
- Most aspects of the MSP432 device can be confured and diven with the DriverLib APIs
- When the high level APIs in the DriverLib are used, powerful code can be written that is highly portable between different families in the MSP430/MSP432 platforms
- Code written using the DriverLib APIs will be more legible and easier to share among a programming team

API Highlights

- API
  - Allow for code portability
  - Makes the code easier to read and understand
- Key idea: The programmer exposes only details that are needed and useful
  - Users of the API aren't burdened by interaction with low-level details
  - Even if the internal details change, the exposed details remain consistent
- Question: Can we go beyond the API-type changes to make the code even more portable and easier to understand?

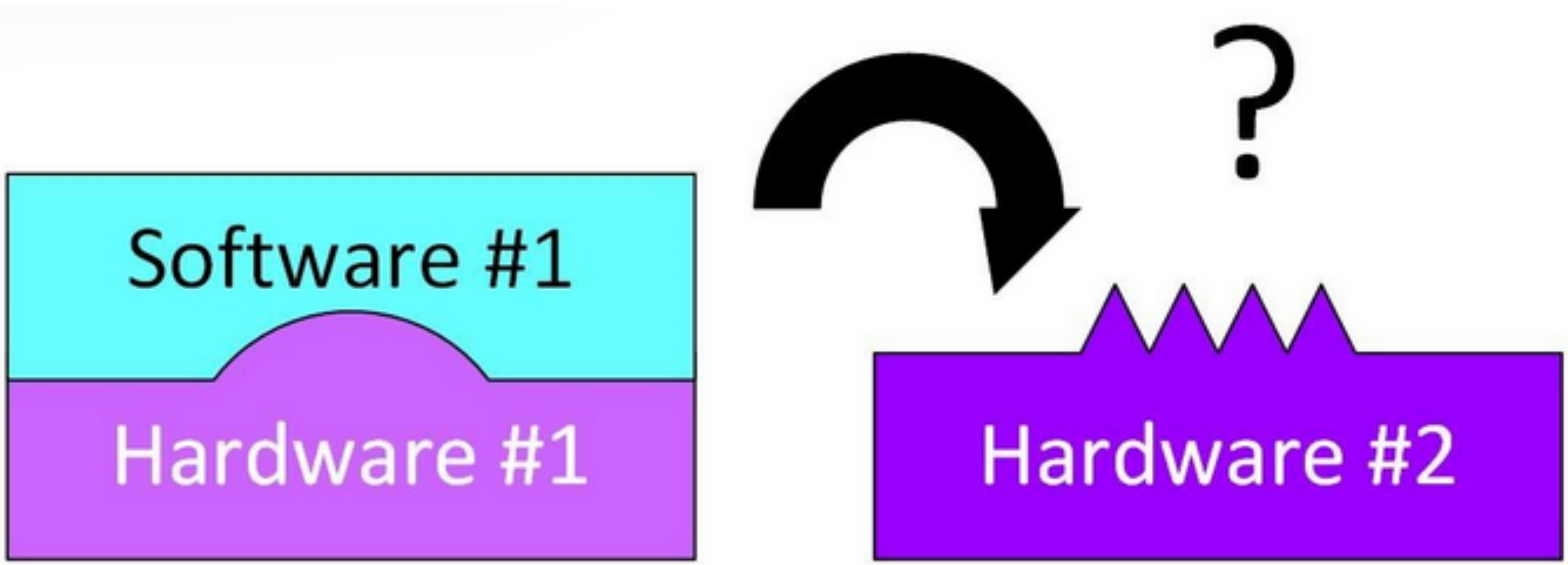
Hardware Abstraction Layer (HAL)

- Hardware Abstraction Layer is a coding approach that places a layer between the software and the hardware
- HAL coding can use device drivers, like DriverLib, but unlike device drivers the code is structured by the programmer

Examples of our Blink code with and without hardware abstraction layers

Software Development for Hardware Platforms

Scenario: You have developed code for a certain microcontroller and now you have to switch to new hardware. Can you move your software to this new system?



Software Development for Hardware Platforms

- Software written for a microcontroller will typically have higher-level functions.
- Depending on how it was developed it can have one or both of the following layers
  - DriverLib functions
  - HAL functions

Software Development for Hardware Platforms



*Driverlib Functions are **NEVER** portable. They are developed to work with a specific hardware.*



*HAL Functions are **NEVER** portable. They are developed to work with a specific hardware or a specific Driverlib associated with a certain hardware.*

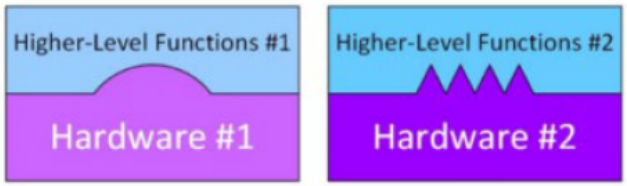


Higher-level functions may be portable if they are stacked on top of HAL functions.

Possible Software Stack Combinations

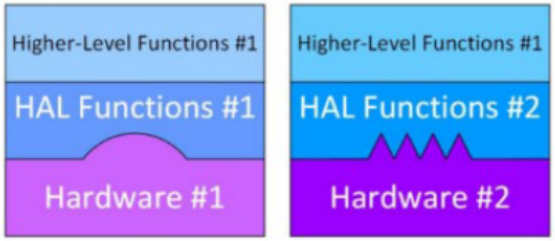
No Driverlib, no HAL

*Higher-level functions are written to work with the specific hardware and therefore, are **NOT** portable!*



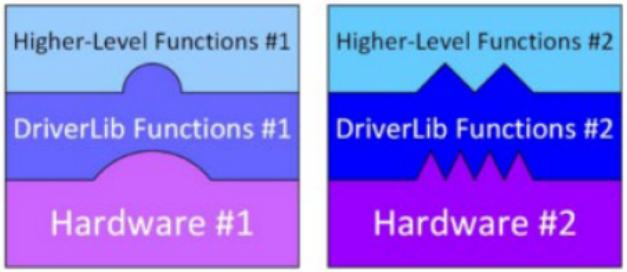
No Driverlib, but HAL

Higher-level functions are written to work with the Hardware Abstraction Layer (HAL) functions. Therefore, they are portable!



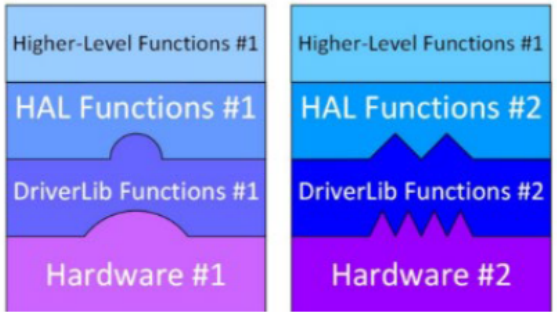
Driverlib, but no HAL

*Higher-level functions are written to work with the specific Driverlib and therefore, are **NOT** portable!*

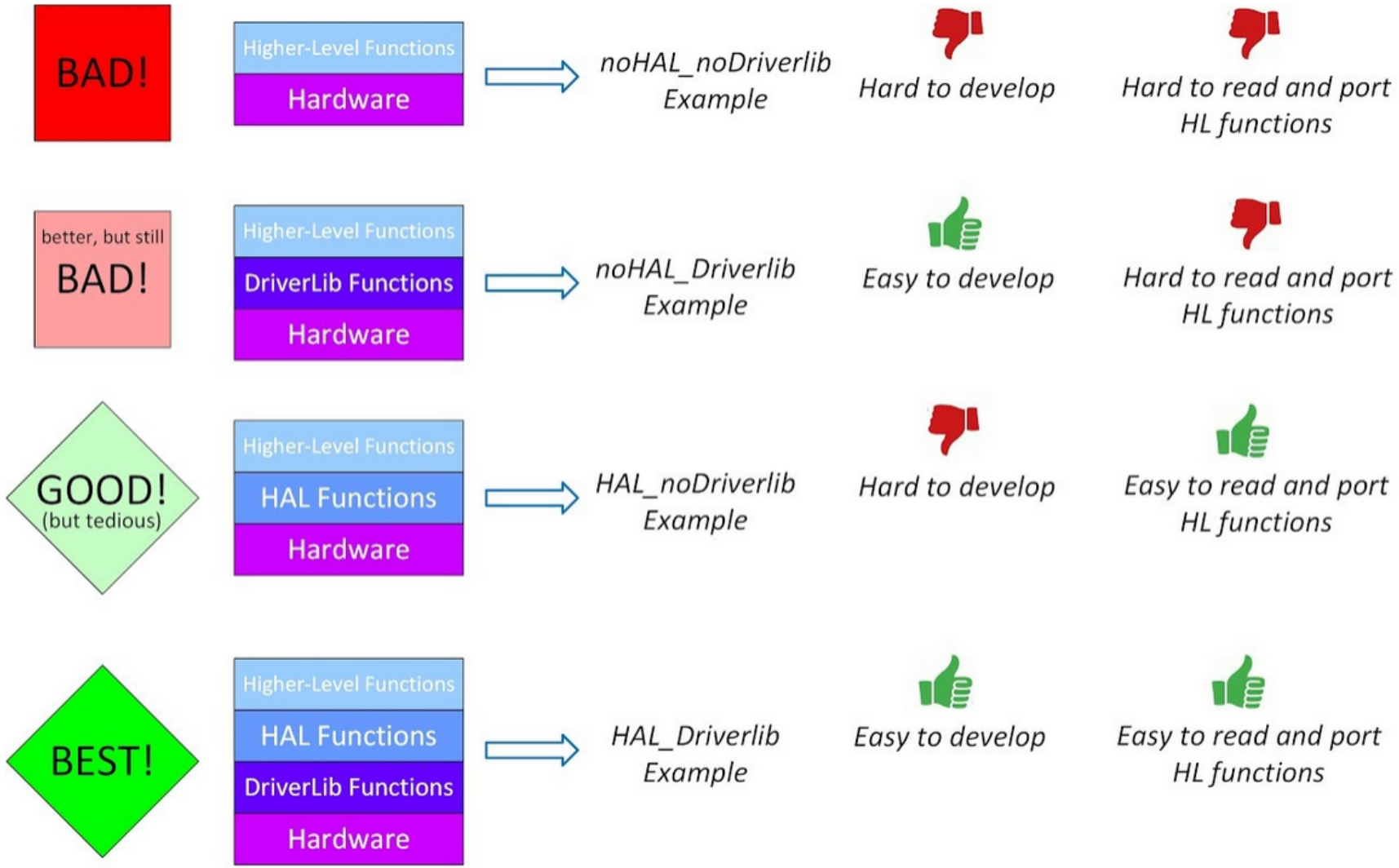


Both Driverlib and HAL

Higher-level functions are written to work with the Hardware Abstraction Layer (HAL) functions. Therefore, they are portable!



Advantages of Combining DriverLib and HAL



Summary

- API: Application Programming Interface
  - HAL: Hardware Abstraction Layer
  - DriverLib: a library of functions provided by TI
- A goal is clean and clear interfacing
  - The application is on top of
  - HAL, which is on top of
  - DriverLib, which is on top of
  - Hardware
- Cost: The API and HAL can add overhead due to the function calls, making the system potentially less efficient